Optimization and reprojection error

2026-02-20

SDU

Get back to a known location

Live demonstration

Use an external webcam to do the following

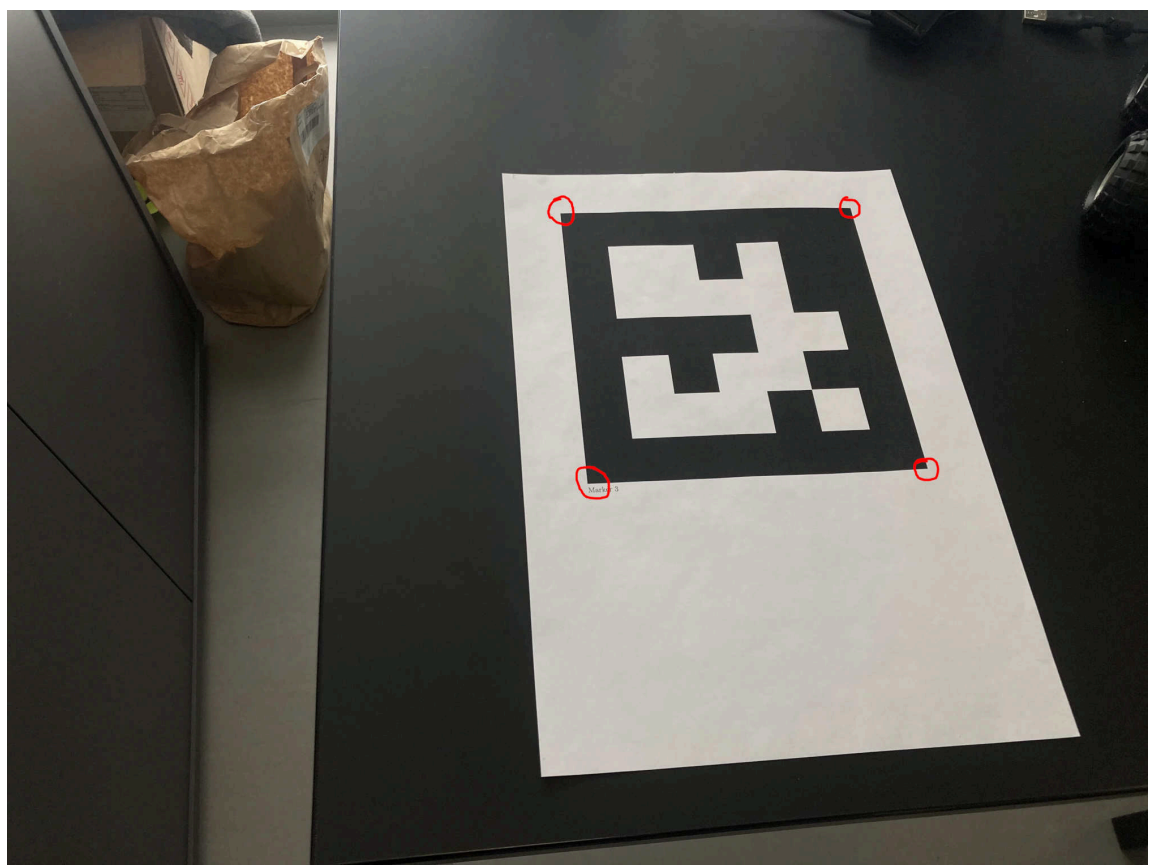
- grab a reference image
- move the camera to a new position
- try to go back to the location where the reference image was taken

```
cd 03_optimization/solutions
uv run python uv run python ex90_demonstration_of_pose_estimation_manual.py
```

Example problem

Example problem

Given an image of an Aruco marker, estimate the camera pose relative to the marker.



Pin hole camera model

$$s \cdot \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \cdot$$

$$\mathrm{ImageCoord} = \mathrm{Internal} \cdot \mathrm{External} \cdot \mathrm{WorldCoord}$$

The internal camera matrix is known

$$\begin{pmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1622.5 & 0. & 1007.9 \\ 0. & 1622.7 & 748.4 \\ 0. & 0. & 1. \end{pmatrix}$$

Projecting points into the camera

Given the

- coordinates of the corners of the marker
- location of the camera
- orientation of the camera
- the internal camera matrix

we can calculate where the points appear on the image.

Python implementation

```
def fProject(self, pose, point_3d):
    Mext = self.externalMatrix(pose)

    point = self.K @ Mext @ point_3d
    point[0, :] = np.true_divide(point[0, :], point[2, :])
    point[1, :] = np.true_divide(point[1, :], point[2, :])
    point = point[0:2, :]

    point = point.reshape((1, -1))

    return point
```

Helper functions - externalMatrix

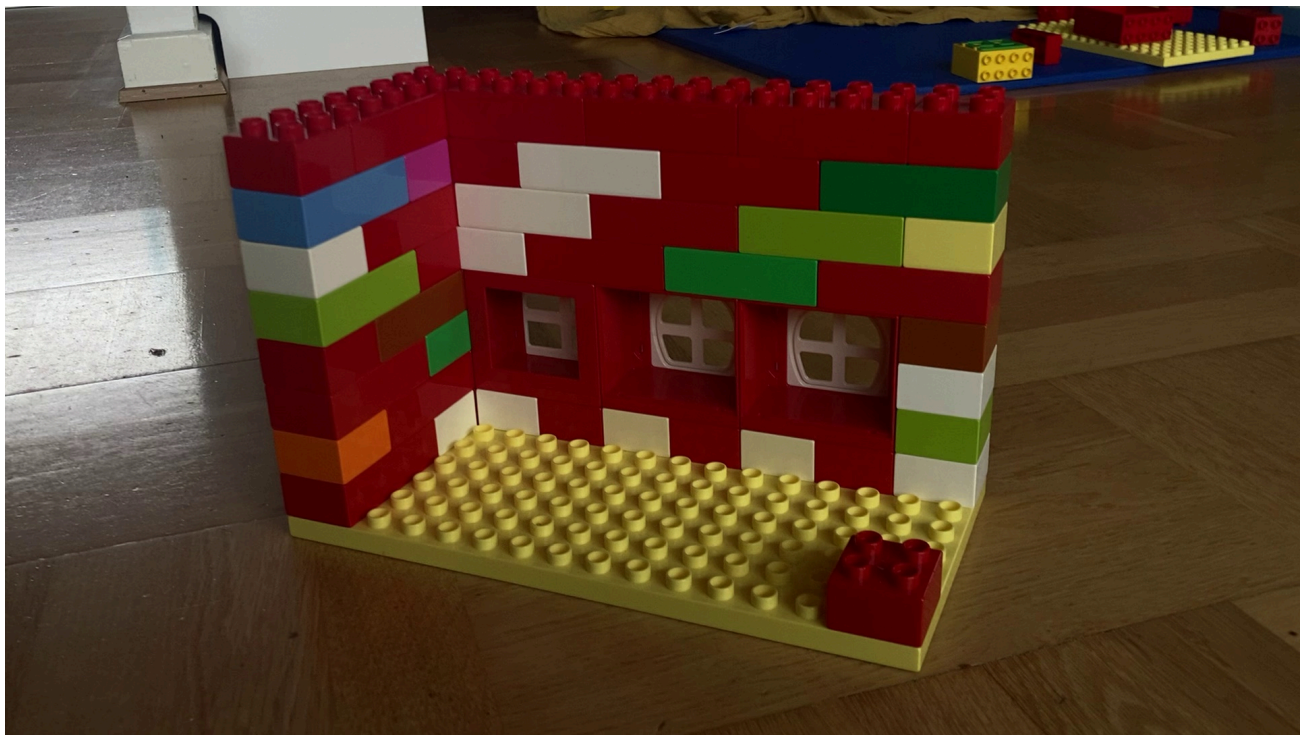
```
def externalMatrix(self, pose):
    ax = pose[0]
    ay = pose[1]
    az = pose[2]
    tx = pose[3]
    ty = pose[4]
    tz = pose[5]

    R = self.rotationMatrix(ax, ay, az)
    Mext = np.concatenate((R, np.array([[tx], [ty], [tz]])), 1)
    return Mext
```

Helper functions - rotationMatrix

```
def rotationMatrix(self, ax, ay, az):
    Rx = np.array([[1, 0, 0], [0, cos(ax), -sin(ax)], [0, sin(ax), cos(ax)]])
    Ry = np.array([[cos(ay), 0, sin(ay)], [0, 1, 0], [-sin(ay), 0, cos(ay)]])
    Rz = np.array([[cos(az), -sin(az), 0], [sin(az), cos(az), 0], [0, 0, 1]])
    R = Rz @ Ry @ Rx
    return R
```

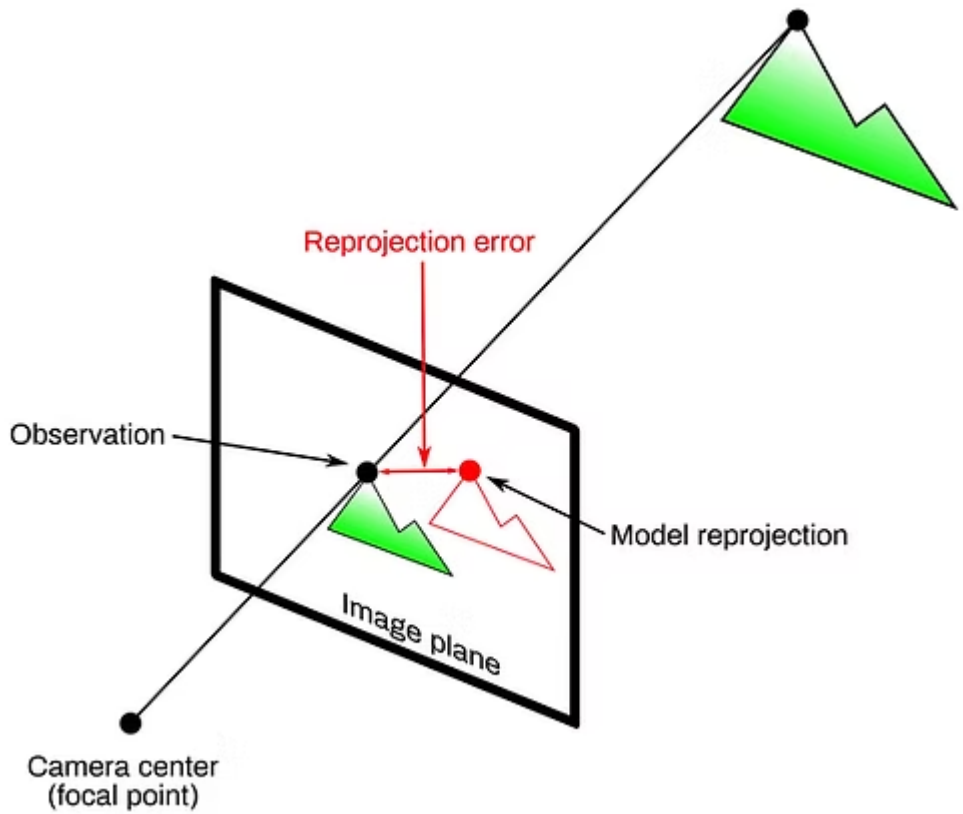
Live demonstration



Try to match match the projected points with a real marker by adjusting the camera pose.

```
cd ~/localbuild/manim-animations
uv run python 85_estimate_camera_pose/estimate_camera_pose_interactive.py
```

Reprojection error



Reprojection error

$$\sum_k \left| p_k - \tilde{p}_k \right|^2$$

Why is it interesting to look at the reprojection error?

- Optimal in a statistical sense

Reprojection error - statistical considerations

Assumption: The image measurement noise is Gaussian distributed

Minimizing the sum of squared reprojection errors is equivalent to finding the most probable model parameters.

Ie. it is the optimal Maximum Likelihood (ML) estimator.

Error measure

Reprojection error, for four points.

$$\vec{r}\left(\beta\right) = \vec{I} - \vec{p}\left(\beta\right) = \begin{bmatrix} I_{1x} \\ I_{2x} \\ I_{3x} \\ I_{4x} \\ I_{1y} \\ I_{2y} \\ I_{3y} \\ I_{4y} \end{bmatrix} - \begin{bmatrix} p_{1x} \\ p_{2x} \\ p_{3x} \\ p_{4x} \\ p_{1y} \\ p_{2y} \\ p_{3y} \\ p_{4y} \end{bmatrix} = \begin{bmatrix} r_{1x} \\ r_{2x} \\ r_{3x} \\ r_{4x} \\ r_{1y} \\ r_{2y} \\ r_{3y} \\ r_{4y} \end{bmatrix}$$

Problem formulation

We would like to minimize the function S, defined as

$$S(\beta) = \vec{r}\,(\beta)^T \;\; \vec{r}\,(\beta) = \sum_{i=1}^{m} r_i(\beta)^2$$

where r_i are the reprojection error functions for each of the known 3D points.

The task is to find the parameters of β that minimizes S.

How to minimize a multivariate function?

Find the gradient and go against it.

$$\beta_{k+1} = \beta_k - h \cdot \nabla S\left(\beta_k\right)$$

h is stepsize.

Linear approximation

$$S\left(\beta + d\beta\right) = S(\beta) + \nabla S(\beta) \cdot d\beta$$

where $\nabla S(\beta)$ is the gradient of S.

Gradient

$$g = \nabla S(\beta) = \begin{bmatrix} \frac{\partial S}{\partial \beta_1} \\ \frac{\partial S}{\partial \beta_2} \\ \frac{\partial S}{\partial \beta_3} \\ \frac{\partial S}{\partial \beta_4} \\ \vdots \end{bmatrix}$$

Gradient from Jacobian

$$g_j = \frac{\partial S}{\partial \beta_j} \qquad\qquad J_{ij} = \frac{\partial r_i}{\partial \beta_j}$$

$$= \frac{\partial}{\partial \beta_j} \sum_{i=1}^{m} r_i(\beta)^2$$

$$= \sum_{i=1}^{m} \frac{\partial}{\partial \beta_j} \left(r_i(\beta)^2 \right)$$

$$= 2 \sum_{i=1}^{m} r_i \cdot \frac{\partial}{\partial \beta_j} r_i(\beta)$$

$$= 2 \sum_{i=1}^{m} r_i \cdot J_{ij}$$

Gradient from Jacobian 2

$$\vec{g} = 2\ J^T \cdot \vec{r}$$

Dimensions of quantities

Number of points in image:

$$n$$

Number of parameters:

$$m$$

Reprojection errors

$$\vec{r}: \quad \langle 2n \times 1 \rangle$$

Jacobi matrix

$$J: \quad \langle 2n \times m \rangle$$

Gradient (of S)

$$\nabla S = \vec{g}: \quad \langle m \times 1 \rangle$$

SDU

Steepest descent

Find the gradient and go against it.

$$\begin{aligned}\beta_{k+1} &= \beta_k - h \cdot \nabla S\left(\beta_k\right) \\ &= \beta_k - h \cdot \vec{g} \\ &= \beta_k - h \cdot 2\ J^T \cdot \vec{r}\end{aligned}$$

Using stepsize h.

It is difficult to fine one good stepsize.

Second order approximation

Second order Taylor expansion of $f(x)$ around x:

$$f(x + \Delta x) \simeq f(x) + f'(x) \cdot \Delta x + \frac{1}{2} \cdot f''(x) \cdot (\Delta x$$

Local peak

Set derivative to zero

$$
\begin{aligned}
0 &= \frac{d}{dx}\left(f(x) + f'(x)\cdot\Delta x + \frac{1}{2}\cdot f''(x)\cdot(\Delta x)^2\right) \\
&= f'(x) + f''(x)\cdot(\Delta x)
\end{aligned}
$$

And solving for Δx

$$
\Delta x = \frac{-f'(x)}{f''(x)}
$$

Update rule

Update rule:

$$
\begin{aligned}
x_{k+1} &= x_k + \Delta x \\
&= x_k - \frac{f'(x_k)}{f''(x_k)}
\end{aligned}
$$

Second order approximation - multivariate

Second order approximation of a multivariate function

$$
\begin{aligned}
A &= f(\vec{x} + \Delta\vec{x}) \\
&\simeq f(\vec{x}) + (\Delta\vec{x}) \cdot \nabla f(\vec{x}) + \frac{1}{2} \cdot (\Delta\vec{x}) \cdot H \cdot (\Delta\vec{x})
\end{aligned}
$$

Local peak - multivariate

$$
\begin{aligned}
0 &= \frac{\partial f}{\partial \Delta \vec{x}} \\
&= \nabla f(\vec{x}) + H \cdot \Delta \vec{x}
\end{aligned}
$$

Solve for $\Delta \vec{x}$

$$
\Delta \vec{x} = -H^{-1} \cdot \nabla f(\vec{x})
$$

Update rule - multivariate

$$\vec{x}_{k+1} = \vec{x}_k - H^{-1} \cdot \nabla f(\vec{x}_k)$$

This is the update rule of the Newton method.

Main issue is that it is difficult / expensive to calculate the Hessian matrix H.

Approximate the Hessian

Why can we make this approximation?

$$H_{kj} = 2\sum_{i=1}^{m}\left(\frac{\partial r_i}{\partial \beta_k}\frac{\partial r_i}{\partial \beta_j} + r_i \cdot \frac{\partial^2 r_i}{\partial \beta_j^2}\right)$$

$$\simeq 2\sum_{i=1}^{m}\frac{\partial r_i}{\partial \beta_k}\frac{\partial r_i}{\partial \beta_j}$$

Close to the solution, the reprojection errors $\vec{r}$ are small.

Puzzle time

$$
\begin{aligned}
S &= \vec{r}^T \cdot \vec{r} \\
\nabla S &= 2\ J^T \cdot \vec{r} \\
H &\simeq 2\ J^T \cdot J \\
\vec{x}_{k+1} &= \vec{x}_k - H^{-1} \cdot \nabla f(\vec{x}_k) \\
&= \vec{x}_k - \left(J^T \cdot J\right)^{-1} \cdot J^T \cdot \vec{r}
\end{aligned}
$$

This is the update rule used in the Gauss Newton algorithm.

Pose estimation – example code

```
def update_pose_estimate(self):
    projected_points, j = self.jacobian()
    points = self.image_points.transpose().reshape((1, -1))
    projection_error = points - projected_points
    self.residual_error = np.linalg.norm(projection_error)
    dx = np.linalg.pinv(j) @ projection_error.reshape((-1, 1))
    self.pose = self.pose + dx.reshape((-1))
```

Pose estimation – Jacobian

```
  def jacobian(self):
      projected_points = self.fProject(self.pose, self.corners)
      e = 0.00001
      j = np.zeros((16, 6))
      j[:, 0] = (self.fProject(self.pose + np.array([e, 0, 0, 0, 0,
0]).transpose(), self.corners) - projected_points) / e
      j[:, 1] = (self.fProject(self.pose + np.array([0, e, 0, 0, 0,
0]).transpose(), self.corners) - projected_points) / e
      j[:, 2] = (self.fProject(self.pose + np.array([0, 0, e, 0, 0,
0]).transpose(), self.corners) - projected_points) / e
      j[:, 3] = (self.fProject(self.pose + np.array([0, 0, 0, e, 0,
0]).transpose(), self.corners) - projected_points) / e
      j[:, 4] = (self.fProject(self.pose + np.array([0, 0, 0, 0, e,
0]).transpose(), self.corners) - projected_points) / e
```

Solved?

Not yet.

Gauss Newton works really well close to the solution.

Demonstation

```
cd ~/localbuild/manim-animations/66_interactive_visualizations
uv run manim show_derivative_mouse_controlled.py --renderer opengl -p
```

The Levenberg Markquardt algorithm

Gauss Newton algorithm

$$\vec{x}_{k+1} = \vec{x}_k - \left(J^T \cdot J\right)^{-1} \cdot J^T \cdot \vec{r}$$

Levenberg Marquardt algorithm

$$\vec{x}_{k+1} = \vec{x}_k - \left(J^T \cdot J + \lambda \cdot I_{3x3}\right)^{-1} \cdot J^T \cdot \vec{r}$$

Levenberg (1944) “A Method for the Solution of Certain Non-Linear Problems in Least Squares”

Fletchers modification

$$\vec{x}_{k+1} = \vec{x}_k - \left(J^T \cdot J + \lambda \cdot \mathrm{diag}\left(J^T \cdot J\right)\right)^{-1} \cdot J^T \cdot \vec{r}$$

What are the benefits / disadvantages of Fletchers modification?

Properties of the algorithms

Steepest descent

- Linear convergence (slow)
- Robust

Gauss-Newton

- Quadratic convergence
- Behaves well near the optima (fragile)

Levenberg-Marquardt

- Best of both worlds
- Quadratic convergence near local optima
- Robust

Pose estimation in practice

2025-04-11 Estimating pose of flower containers - slides.pdf

Outro

Course outline

2026-02-01

- Python and 2D image processing
- Camera in space
- Optimization

2026-03-01

- Fiducial markers
- Camera calibration mini project
- Feature detectors
- Motion detection mini project
- Orthomosaics

2026-04-01

- Pumpkin counting mini project
- Visual odometry
- Visual odometry mini project

2026-05-01

- Bag of Visual Words / AI overview

Quote

‘’One of my most productive days was throwing away 1000 lines of code.’’

Ken Thompson